

Supplementary Material for “Queryable Semantic Compression for Large-scale Knowledge Graphs via Differentiable Learning”

Junhua Ma
Sun Yat-sen University
Guangzhou, China
Peng Cheng Laboratory
Shenzhen, China
majh33@mail2.sysu.edu.cn

Kunxun Qi
The Hong Kong University of Science
and Technology (Guangzhou)
Guangzhou, China
kunxunqi@foxmail.com

Hai Wan
Sun Yat-sen University
Guangzhou, China
wanhai@mail.sysu.edu.cn

Yue Yu
Peng Cheng Laboratory
Shenzhen, China
yuy@pcl.ac.cn

Wei Wang
The Hong Kong University of Science
and Technology (Guangzhou)
Guangzhou, China
weiwcs@ust.hk

PVLDB Reference Format:

Junhua Ma, Kunxun Qi, Hai Wan, Yue Yu, and Wei Wang. Supplementary Material for “Queryable Semantic Compression for Large-scale Knowledge Graphs via Differentiable Learning”. PVLDB, 20(1): XXX-XXX, 2027. doi:XX.XX/XXX.XX

1 STORED RULE-LEARNER PARAMETERIZATION

Equation (5) defines the logical probabilities $p_{r,j,\ell,\rho}$ consumed by rule execution. MiniKG does not store an independent scalar for every tuple (r, j, ℓ, ρ) . Training runs one bidirectional LSTM per candidate rule over a shared channel-embedding sequence: an embedding $e_\rho \in \mathbb{R}^d$ for each channel $\rho \in \mathcal{R}$, one bidirectional LSTM for each of the L candidate rules, and a shared affine output layer over $\mathcal{R}$ turn the embedding of a target relation r , repeated for the T rule steps and terminated by the identity-channel embedding, into logits that are normalized across relation channels to obtain $p_{r,j,\ell,\rho}$. At serialization, however, each candidate rule stores whichever of its two forms is smaller: the rule’s LSTM weights, or the rule’s decoded probabilities directly, which form a $T \times K$ matrix with $K = |\mathcal{R}| = 2|\mathcal{R}_0| + 1$. Inverse-target probabilities use the inverse construction stated in the main paper rather than a second stored model.

With LSTM input and hidden width d , one rule’s LSTM weights count $16d^2 + 16d$ parameters, so the stored rule-learner parameter count is

$$|\theta| = Kd + \sum_{j=1}^L \min(16d^2 + 16d, TK) + (2Kd + K), \quad (S1)$$

where the terms respectively count the channel embeddings, the L per-rule minima, and the shared affine output layer. Thus stored rule parameters scale as $O(Kd + L \min(d^2, TK))$, and a rule is stored

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

by its LSTM weights exactly when those weights are the smaller form. The serialized ratio charges the stored tensors in Equation (S1) to $B(\theta)$; it does not charge a hypothetical dense $|\mathcal{R}_0| \times T \times L \times K$ table.

Retention logits z are training state rather than part of the finalized representation. Finalization binarizes them into the retained fact set S , after which the serialized representation contains S , θ , the decode policy π , exact residuals M , and correction terms X .

2 EXPERIMENTAL ACCOUNTING AND QUERY PROTOCOL

2.1 Compression resources

Compression time is end-to-end wall-clock time over construction and finalization. Peak combined resident memory is the maximum, over all samples in that interval, of host resident set size plus GPU-resident memory; the GPU term is zero for CPU-only runs. GPU runs use 104 GiB of host memory and one NVIDIA RTX 4090 with 24 GiB of device memory, while CPU-only runs use a 128-GiB host with an Intel Xeon Gold 6338N processor. Segmented symbolic baselines receive four hours for rule mining and four hours for compression, and MiniKG receives eight hours end to end.

2.2 Workloads

Template workloads. Family, CoDEx-S/M/L, YAGO3-10, and Wiki-data5M use workloads generated from the sixteen multi-hop templates of Query2Box [3] and BetaE [4], which instantiate the query language of Equation (1). Table S1 lists the sixteen templates and their per-template query counts: the seven templates without negation or union contribute 100 queries each and the nine remaining templates contribute 80 each, giving 1,420 queries per graph. In the template names, p denotes a projection hop, i an intersection, n a branch negated through set difference, and u a union, so 3p is a three-hop projection chain and 2in is a two-way intersection with one negated branch. The DNF and DM suffixes denote two evaluation forms of the same union structure, one applying the union directly and one applying its De Morgan rewriting. Every query is generated by sampling a start entity and walking the template on the original graph, and only queries with a non-empty answer are

Table S1: Composition of the sixteen-template workload. Each template splits its instances evenly between forward and backward relation reads.

Template group	Templates	Queries each
Projection chains	1p, 2p, 3p	100
Intersections	2i, 3i	100
Chains with intersections	ip, pi	100
Negations	2in, 3in, inp, pin, pni	80
Unions	2u-DNF, 2u-DM, up-DNF, up-DM	80

kept. Within every template, half of the instances read their relations forward and half backward, and the workload concatenates the templates in a fixed order.

Freebase workloads. Freebase uses SimpleQuestions [1], GrailQA [2], WebQSP [6], and ComplexWebQuestions [5] as four separately reported workloads. Each question is executed as the relation-path query that its benchmark releases with it, and the executed workloads contain 94,538, 37,221, 4,491, and 22,796 queries respectively. SimpleQuestions consists entirely of single-relation projections, GrailQA mixes single projections with two- and three-hop chains, and WebQSP and ComplexWebQuestions pose multi-hop queries whose intermediate projections fan out widely.

2.3 Measured and modeled query cost

Each warm-workload bar in Figure 6 is the mean of repetitions 2–6 after one warm-up execution at the same batch size. The four measured batch sizes are 1, 64, 128, and 1024. The cumulative curves in Figure 7 are modeled from measured start-up and warm times as

$$T(N) = T_{\text{start}} + N T_{\text{warm}}. \quad (\text{S2})$$

3 PROOFS

For completeness, we first restate the five results proved below.

Lemma 1 (Local recovery decomposition). For every $r \in \mathcal{R}$ and $u \in \mathcal{E}$, $D_r(S, \theta, \pi)[u] = D_r(u)$.

Lemma 2 (Relation exactness). For every $r \in \mathcal{R} \setminus \mathcal{R}_{\text{full}}$, $(\widehat{G}_r \cup M_r) \setminus X_r = G_r$.

Theorem 1 (Lossless reconstruction). For every $S \subseteq G$, θ , and π such that $G_r \subseteq S$ for every $r \in \mathcal{R}_{\text{full}}$, the finalized representation $C = (S, \theta, \pi, M, X)$ satisfies $\text{CorrectedDecode}(C) = G$.

Lemma 3 (Local projection exactness). For every $u \in \mathcal{E}$ and $r \in \mathcal{R}$, $A_C(u, r) = G_r[u]$; consequently, $\text{project}_C(U, r) = \text{project}_G(U, r)$ for every $U \subseteq \mathcal{E}$.

Theorem 2 (Compositional query equivalence). Every query Q in the grammar of Equation (1) of the main paper satisfies $\text{Ans}_C(Q) = \text{Ans}_G(Q)$.

PROOF OF LEMMA 1. By definition,

$$D_r(S, \theta, \pi) = \bigcup_{a \in \mathcal{E}} \{(a, r, v) : v \in D_r(a)\}.$$

Taking the slice whose first argument is u removes every union term with $a \neq u$ and leaves exactly $\{v : v \in D_r(u)\} = D_r(u)$. $\square$

PROOF OF LEMMA 2. Substituting the definitions of M_r and X_r gives

$$(\widehat{G}_r \cup M_r) \setminus X_r = (\widehat{G}_r \cup (G_r \setminus \widehat{G}_r)) \setminus (\widehat{G}_r \setminus G_r).$$

The union contains every element of G_r , and the subtracted set contains exactly its elements outside G_r ; the result is G_r . $\square$

PROOF OF THEOREM 1. We show $(\widehat{G} \cup M) \setminus X = G$. Every residual atom is a true fact, $M_r \subseteq G_r$, and every correction atom is a false one, $X_r \cap G_r = \emptyset$; reading an inverse-direction atom as its forward triple preserves both properties, so $M \subseteq G$ and $X \cap G = \emptyset$. For one inclusion, let $x \in G$ have original relation r . If $r \in \mathcal{R}_{\text{full}}$, the premise $G_r \subseteq S$ puts x in the S_r part of $\widehat{G}$; otherwise Lemma 2 gives $x \in \widehat{G}_r \cup M_r$. Since $X \cap G = \emptyset$, x survives the subtraction, so $G \subseteq (\widehat{G} \cup M) \setminus X$. For the other, let $y \in \widehat{G} \cup M$ with $y \notin G$ and original relation r . Then $y \notin M$ because $M \subseteq G$, and $y \notin S_r$ because $S \subseteq G$, so $r \notin \mathcal{R}_{\text{full}}$ and $y \in \widehat{G}_r \setminus G_r = X_r \subseteq X$, and y is removed. Hence $\text{CorrectedDecode}(C) = G$. $\square$

PROOF OF LEMMA 3. If $r \in \mathcal{R}_{\text{full}}$, then S contains the full slice of r 's original relation, and reading it in r 's direction gives $A_C(u, r) = S_r[u] = G_r[u]$. If $r \notin \mathcal{R}_{\text{full}}$, Lemma 1 gives $D_r(u) = D_r(S, \theta, \pi)[u]$, and the u -slice of Lemma 2, which is stated for every $r \in \mathcal{R} \setminus \mathcal{R}_{\text{full}}$ and thus covers both directions, yields $(D_r(u) \cup (S_r \cup M_r)[u]) \setminus X_r[u] = G_r[u]$. Therefore $A_C(u, r) = G_r[u]$ in both cases, and union over $u \in U$ gives $\text{project}_C(U, r) = \text{project}_G(U, r)$. $\square$

PROOF OF THEOREM 2. Proceed by structural induction on Q . For the base expression $Q = H$, both evaluators return the entity set H . For $Q = \text{Proj}_r[Q_0]$, the induction hypothesis and Lemma 3 give $\text{Ans}_C(Q) = \text{project}_C(\text{Ans}_C(Q_0), r) = \text{project}_G(\text{Ans}_C(Q_0), r) = \text{project}_G(\text{Ans}_G(Q_0), r) = \text{Ans}_G(Q)$. For a union, intersection, or difference expression, the induction hypotheses make the corresponding operand answers identical, and the same set operation preserves equality. These cases exhaust Equation (1). $\square$

REFERENCES

- [1] Antoine Bordes, Nicolas Usunier, Sumit Chopra, and Jason Weston. 2015. Large-scale Simple Question Answering with Memory Networks. *arXiv preprint arXiv:1506.02075* (2015).
- [2] Yu Gu, Sue Kase, Michelle Vanni, Brian Sadler, Percy Liang, Xifeng Yan, and Yu Su. 2021. Beyond I.I.D.: Three Levels of Generalization for Question Answering on Knowledge Bases. In *WWW*. 3477–3488.
- [3] Hongyu Ren, Weihua Hu, and Jure Leskovec. 2020. Query2box: Reasoning over Knowledge Graphs in Vector Space Using Box Embeddings. In *ICLR*.
- [4] Hongyu Ren and Jure Leskovec. 2020. Beta Embeddings for Multi-Hop Logical Reasoning in Knowledge Graphs. In *NeurIPS*. 19716–19726.
- [5] Alon Talmor and Jonathan Berant. 2018. The Web as a Knowledge-Base for Answering Complex Questions. In *NAACL*. 641–651.
- [6] Wen tau Yih, Matthew Richardson, Christopher Meek, Ming-Wei Chang, and Jina Suh. 2016. The Value of Semantic Parse Labeling for Knowledge Base Question Answering. In *ACL*. 201–206.